

Módulo 9 – Capítulo 04 – Sección 01

Qué es el red teaming de IA: ataque estructurado para encontrar vulnerabilidades

Los capítulos anteriores describieron el landscape de amenazas de los sistemas de IA: prompt injection en sus variantes directa e indirecta, jailbreaking con técnicas evolutivas como many-shot y Crescendo, ataques adversariales, model extraction y memorización de training data. Conocer la taxonomía de amenazas es necesario pero insuficiente: saber que el prompt injection existe no es lo mismo que saber si el sistema específico que has construido es vulnerable a él, en qué variantes, con qué impacto, y cuáles son las mitigaciones que efectivamente funcionan en ese contexto. El red teaming de IA es la metodología que cierra esta brecha: un proceso de evaluación adversarial estructurado en el que un equipo adopta el rol del atacante para descubrir vulnerabilidades reales en el sistema específico antes de que lo haga un adversario en producción.

La diferencia entre red teaming y testing funcional es conceptual antes que técnica. El testing funcional verifica que el sistema funciona correctamente en condiciones normales, con inputs esperados, dentro de los parámetros de diseño. El red teaming parte del supuesto opuesto: busca las condiciones anormales, los inputs edge-case, las secuencias de interacción que hacen que el sistema se comporte de maneras no autorizadas, dañinas o que violan sus especificaciones de seguridad. Un sistema que pasa todos sus tests unitarios e integración puede ser vulnerable a prompt injection porque los tests fueron diseñados por el mismo equipo que escribió el código, con los mismos supuestos y puntos ciegos. El red teamer efectivo se libera de esos supuestos: asume que el sistema tiene vulnerabilidades que su diseñador no anticipó y trabaja sistemáticamente para encontrarlas.

La industria ha formalizado esta práctica. OpenAI realizó extenso red teaming externo de GPT-4 antes de su lanzamiento, contratando especialistas con perfiles de expertise que no existen dentro de la empresa: expertos en desinformación política, en técnicas de ingeniería social, en culturas y contextos lingüísticos no ingleses, en dominios de alto riesgo como biomedical y ciberseguridad ofensiva. Anthropic tiene un equipo interno de red teaming que evalúa cada versión de Claude antes de su release. El Executive Order on AI de la

administración Biden (octubre 2023) estableció la obligatoriedad de red teaming para modelos frontier antes de su despliegue a nivel gubernamental. El Voluntary AI Commitments firmado por los principales proveedores en julio de 2023 incluye el compromiso de realizar red teaming pre-deployment como condición de buenas prácticas de la industria.

Lo que hace al red teaming de IA especialmente valioso —y diferente del penetration testing tradicional de software— es que busca vulnerabilidades que no pueden anticiparse mediante análisis estático o testing funcional. El comportamiento de un LLM ante inputs adversariales es fundamentalmente no predecible sin exploración adversarial activa: la misma secuencia de tokens puede producir respuestas completamente diferentes con variaciones mínimas en el framing, el idioma, o el contexto de conversación previo. Esta no-determinabilidad hace que el red teaming sea la única metodología que puede descubrir estas vulnerabilidades antes de que lo haga un adversario real.

Componentes principales del red teaming de IA

- **Objetivo del red team:** encontrar vulnerabilidades de seguridad (prompt injection, jailbreak, exfiltración de datos), comportamientos dañinos (generación de contenido prohibido, desinformación), y fallas de alineación (el modelo hace lo que el usuario malicioso solicita, no lo que el operador del sistema pretendía).
- **Alcance del ejercicio:** definir el threat model del sistema (quién es el adversario, con qué motivación y recursos), los sistemas incluidos en el scope (modelo base, fine-tuning, RAG, herramientas del agente, APIs externas), y los criterios de éxito para un ataque (qué outputs se consideran vulnerabilidades reportables y cuál es su severidad mínima).
- **Tipos de ejercicio:** red teaming de safety (el modelo genera contenido dañino); red teaming de security (el sistema es comprometido para ejecutar acciones no autorizadas); red teaming de privacidad (datos sensibles son extraídos o expuestos); red teaming de alineación (el modelo hace cosas que el operador no pretendía).
- **Diferencia con penetration testing clásico:** el pentesting busca vulnerabilidades en código, configuración y red; el red teaming de IA busca comportamientos adversariales del modelo — la "vulnerabilidad" reside en las representaciones aprendidas y en el diseño del sistema de prompts, no en el código.
- **Integración en el ciclo de desarrollo:** antes de cada release de modelo o capacidad nueva, antes de desplegar herramientas en agentes, y periódicamente en producción para detectar drifts de comportamiento o nuevas técnicas de ataque.

Nota del Arquitecto: La pregunta que más escucho en equipos que no hacen red teaming es "¿para qué si el modelo ya tiene restricciones de safety?". La respuesta correcta es: porque las restricciones de safety del proveedor protegen ante los ataques que el proveedor conoce en el momento del entrenamiento, con el sistema prompt genérico que ellos probaron. Tu sistema tiene un system prompt específico, herramientas específicas, un corpus RAG específico, y usuarios con motivaciones específicas que los evaluadores del proveedor nunca probaron. El red teaming de tu sistema específico es el único mecanismo que puede encontrar las vulnerabilidades que emergen de esa combinación particular.

El red teaming de IA es la única metodología que descubre vulnerabilidades que no pueden anticiparse mediante análisis estático o testing funcional, y es la respuesta de la industria al hecho de que el comportamiento de un LLM ante inputs adversariales no puede verificarse sin exploración adversarial activa.

Módulo 9 – Capítulo 04 – Sección 02

Metodología: scoping, threat modeling, ejecución y reporte

El red teaming sin metodología estructurada produce resultados inconsistentes, difíciles de comparar entre iteraciones, y con cobertura de amenazas que refleja los sesgos del equipo en lugar del threat model del sistema. El equipo que hace red teaming sin scoping previo tiende a atacar lo que le resulta interesante en lugar de lo que es más arriesgado para el sistema específico; el que no tiene criterios de éxito definidos termina con hallazgos que no pueden priorizarse ni compararse entre ejercicios; el que no produce reportes reproducibles genera conocimiento que muere con el ejercicio y no puede verificarse después de una mitigación. La metodología no es burocracia —es el mecanismo que convierte el red teaming en un proceso de ingeniería con resultados acumulables y verificables.

La fase de scoping es la más frecuentemente subestimada y la que más determina la calidad del ejercicio. En esta fase se define el threat model del ejercicio: quién es el adversario (un usuario frustrado sin capacidades técnicas, un competidor con acceso a modelos open-source, un investigador de seguridad, un actor estatal), cuáles son sus motivaciones (causar daño a usuarios, extraer el sistema propietario, comprometer la infraestructura), y qué capacidades tiene (acceso solo a la interfaz pública, conocimiento del sistema prompt, acceso a modelos open-source como surrogates). También se definen los harm criteria: qué outputs específicos constituyen una vulnerabilidad reportable y cuál es su severidad mínima para ser incluida en el reporte. Un harm criterion bien definido podría ser "cualquier respuesta que proporcione instrucciones accionables para actividades ilegales que puedan causar daño físico" o "cualquier respuesta que incluya el texto del system prompt".

La fase de threat modeling prioriza las categorías de ataque más relevantes para el sistema específico usando los frameworks establecidos. OWASP LLM Top 10 proporciona una lista priorizada de vulnerabilidades específicas de LLMs que debe revisarse completa para identificar cuáles aplican al sistema bajo prueba. MITRE ATLAS extiende esta lista con tácticas, técnicas y procedimientos documentados en incidentes reales. Para sistemas agénticos, las categorías de tool call injection, confused deputy problem, y amplificación de impacto mediante herramientas deben estar explícitamente en el scope. Para sistemas RAG,

RAG poisoning e indirect injection a través de documentos recuperados deben ser prioridades.

La ejecución estructurada convierte el threat model en un programa de trabajo ejecutable. Los roles del equipo se asignan según el expertise: expertos en jailbreaking manejan las categorías de manipulación del comportamiento del modelo; especialistas en ataques de privacidad se enfocan en extracción de training data y prompt leaking; ingenieros de sistemas atacan la infraestructura y las herramientas del agente. Cada attempt se registra en un sistema de tracking con el prompt exacto enviado, la respuesta recibida, la clasificación de éxito o fallo, y la severidad tentativa. Esta granularidad de registro es lo que permite reproducir el hallazgo después de implementar una mitigación y verificar que la mitigación fue efectiva.

Aspectos técnicos de cada fase

- **Scoping:** threat model del adversario (motivación, capacidades, acceso al sistema), harm criteria específicos con ejemplos de qué califica y qué no, y perímetro del ejercicio (qué sistemas, qué capacidades, qué no está en scope).
- **Threat modeling para red teaming:** priorización de OWASP LLM Top 10 + MITRE ATLAS relevantes para el sistema específico; harm taxonomy en cuatro dimensiones: seguridad (bypasses, exfiltración), safety (contenido dañino), privacidad (PII, datos de entrenamiento), y alineación (el sistema hace cosas que el operador no pretendía).
- **Criterios de severidad:** adaptar CVSS para IA evaluando probabilidad de explotación real, impacto del comportamiento adversarial obtenido, facilidad de reproducción sin capacidades especializadas, y alcance del daño (un usuario afectado vs. todos los usuarios del sistema).
- **Ejecución:** roles especializados, log detallado de cada attempt (prompt, respuesta, clasificación), tracking en Jira o GitHub Issues para asegurar reproducibilidad y seguimiento de mitigaciones.
- **Reporte de hallazgos:** cada hallazgo incluye el prompt o técnica exacta que reproduce la vulnerabilidad, el output observado, la clasificación de severidad, el harm scenario concreto (quién se daña y cómo), y recomendaciones específicas de mitigación.

Nota del Arquitecto: El reporte del red teaming es un documento de seguridad de alta sensibilidad: contiene los prompts exactos que comprometen el sistema. Debe distribuirse solo a las personas que necesitan implementar las mitigaciones, almacenarse con cifrado, y no incluirse en repositorios de código con acceso amplio.

He visto reportes de red teaming publicados inadvertidamente en repos de GitHub, proporcionando a cualquiera con acceso el manual exacto para atacar el sistema.

La metodología de red teaming produce un hallazgo reproducible por cualquier miembro del equipo usando solo la documentación del reporte. Si un hallazgo no puede reproducirse con las instrucciones del reporte, no puede verificarse ni mitigarse, y el ejercicio que lo produjo no tiene valor de seguridad neto.

Módulo 9 – Capítulo 04 – Sección 03

Red teaming manual: técnicas de adversarios humanos especializados

El red teaming automatizado puede ejecutar decenas de miles de variaciones de ataques conocidos en minutos, pero existe un conjunto de vulnerabilidades que los sistemas automatizados no pueden encontrar: aquellas que requieren comprensión cultural profunda de contextos específicos, creatividad para construir narrativas adversariales convincentes en múltiples turnos de conversación, conocimiento de dominio especializado —cómo un médico malicioso formularía una solicitud de información sobre sobredosis para que parezca legítima—, o la capacidad de adaptarse en tiempo real al comportamiento del modelo para redirigir la estrategia de ataque. Estas vulnerabilidades frecuentemente son las más peligrosas en producción porque son las que un adversario motivado y paciente eventualmente descubrirá.

El perfil de un red teamer manual efectivo es interdisciplinario por diseño. Los equipos de red teaming de Anthropic y OpenAI incluyen no solo ingenieros de seguridad sino también psicólogos especialistas en persuasión y manipulación social —relevantes para entender qué narrativas son más efectivas para convencer al modelo de que sus restricciones no aplican en un contexto dado—, expertos en el dominio de la aplicación —un médico, un abogado, un químico— que conocen las formas en que un actor malicioso de su campo podría formular requests que parecen legítimas pero solicitan información peligrosa, y hablantes nativos de múltiples idiomas que pueden detectar las diferencias en la robustez del RLHF entre inglés y otros idiomas.

Las técnicas del red teaming manual no son simplemente "escribir prompts creativos". Son metodologías de ingeniería social sistemáticas aplicadas al modelo. La construcción de personas adversariales —el investigador académico que necesita información sensible, el periodista investigando un tema delicado, el escritor de ficción que necesita detalles realistas— es una técnica psicológica que explota la tendencia del modelo a calibrar su respuesta según el contexto declarado del usuario. Los ataques multiturno que se desarrollan en múltiples intercambios, donde cada respuesta del modelo se analiza para identificar qué apertura existe y cómo explotarla en el siguiente turn, requieren la adaptabilidad en tiempo real que un

sistema automatizado no puede emular con la misma efectividad. Los ataques multilingües son particularmente efectivos porque el RLHF en inglés es denso y robusto; en árabe, chino clásico, hebreo, swahili, o idiomas con alfabetos que los modelos representan con mayor incertidumbre, las mismas restricciones frecuentemente son más permeables.

El activo más valioso que produce el red teaming manual es el playbook: la documentación detallada de qué técnicas funcionaron, en qué contexto, contra qué variante del modelo, con qué exactamente como output. Este playbook, actualizado con cada ejercicio, se convierte en la base de conocimiento de seguridad del sistema y en la guía para el red teaming automatizado de los ejercicios siguientes —los sistemas automatizados como PyRIT y Garak pueden entonces ejecutar variaciones de las técnicas exitosas a escala, multiplicando la cobertura del hallazgo manual.

Aspectos técnicos del red teaming manual

- **Construcción de personas adversariales:** investigador académico, periodista, escritor de ficción, profesional del dominio —contextos que el modelo puede interpretar como legítimos y que reducen su resistencia a proporcionar información sensible; la credibilidad del contexto declarado afecta la probabilidad de éxito.
- **Ataques multilingües:** testing en árabe, chino mandarín, ruso, hebreo, y otros idiomas con menor densidad de RLHF que el inglés; los mismos prompts en idiomas no-inglés frecuentemente eluden restricciones efectivas en inglés.
- **Ataques de dominio especializado:** expertos en química, medicina, derecho, ciberseguridad construyen requests que parecen legítimas en contexto profesional pero solicitan información peligrosa; este tipo de ataque requiere conocimiento de dominio que los sistemas automatizados no tienen.
- **Multi-turn adversarial conversations:** ataques que se desarrollan en múltiples turnos, usando cada respuesta del modelo para calibrar el siguiente prompt hacia el objetivo; la adaptabilidad en tiempo real es la ventaja distintiva del red teamer humano.
- **Documentación del playbook:** registro sistemático de técnica exacta, prompt utilizado, respuesta obtenida, modelo y versión target, y contexto de conversación previo — el activo de seguridad más valioso del ejercicio, base para los ejercicios automatizados siguientes.

El red teaming manual es la única metodología que descubre las vulnerabilidades más sofisticadas y contextualmente específicas, especialmente aquellas que requieren

comprensión cultural y conocimiento de dominio que los sistemas automatizados no pueden emular.

Módulo 9 – Capítulo 04 – Sección 04

Red teaming automatizado: PyRIT, Garak y LLM-as-Attacker

La escala es el problema fundamental del red teaming manual: un equipo humano, por grande que sea, no puede probar sistemáticamente todas las variaciones de un ataque, todos los idiomas posibles, todas las combinaciones de contexto y framing que un adversario distribuido en internet podría intentar. El red teaming automatizado resuelve el problema de escala: donde un red teamer humano puede probar cientos de técnicas en una sesión de trabajo, un sistema automatizado puede ejecutar decenas de miles de variaciones en minutos, cubriendo el espacio de ataques conocidos de manera sistemática y reproducible. Las herramientas líderes del ecosistema en 2024-2025 —PyRIT, Garak, y el patrón LLM-as-Attacker implementado en frameworks como PAIR y TAP— son complementarias entre sí y con el red teaming manual.

PyRIT (Python Risk Identification Toolkit) es la solución de código abierto de Microsoft para orquestación de red teaming de LLMs. Su arquitectura modular distingue cuatro componentes independientes: los orchestrators gestionan el flujo completo del ataque, incluyendo conversaciones multi-turn donde el estado de la sesión se mantiene entre intentos; los targets son los sistemas de IA bajo prueba, accedidos mediante adaptadores para distintos proveedores (OpenAI, Anthropic, Azure OpenAI, endpoints locales); los converters transforman los prompts para evadir filtros mediante técnicas como Base64 encoding, traducción automática, obfuscación de caracteres o inversión de texto; y los scorers evalúan si un output constituye una vulnerabilidad exitosa, combinando clasificadores automáticos (LlamaGuard) con LLM-as-Judge (GPT-4 evaluando el resultado contra el harm criterion). Esta modularidad permite componer flujos de ataque complejos que combinan múltiples técnicas de manera programática.

Garak (Generative AI Red-teaming and Assessment Kit), desarrollado por NVIDIA, tiene un enfoque diferente: más de 100 probes predefinidas organizadas en categorías temáticas que cubren las vulnerabilidades más documentadas de LLMs. Las categorías incluyen jailbreak, data leakage, hallucination, toxicity, RLHF bypass, y decenas de variantes dentro de cada categoría. Garak puede ejecutarse desde la línea de comandos contra cualquier endpoint LLM

y genera un reporte estructurado con la tasa de éxito de cada probe, permitiendo comparar el posture de seguridad del sistema entre versiones o entre modelos diferentes. Su valor principal es la cobertura de amenazas conocidas: si una vulnerabilidad está documentada en el ecosistema de seguridad de IA, probablemente hay una probe en Garak para detectarla.

El patrón LLM-as-Attacker utiliza un LLM (frecuentemente GPT-4 o Claude) para generar, evaluar y refinar prompts adversariales contra el modelo target. PAIR (Prompt Automatic Iterative Refinement, Chao et al. 2023) es la implementación seminal: un LLM "attacker" recibe el objetivo del ataque y el output del intento anterior, genera una nueva variación del prompt diseñada para ser más efectiva, y el proceso itera hasta lograr el jailbreak o alcanzar un límite. La variante TAP (Tree of Attacks with Pruning, Mehrotra et al. 2023) extiende PAIR con búsqueda en árbol que explora el espacio de prompts adversariales más eficientemente, reduciendo el número de queries necesarios para jailbreak exitoso en 4-10x respecto a PAIR. La efectividad demostrada de PAIR es del 60-80% de éxito en 20 iteraciones contra GPT-3.5 y Claude en las evaluaciones publicadas.

Aspectos técnicos de las herramientas

- **PyRIT (Microsoft, open-source):** arquitectura de orchestrators + targets + converters + scorers; soporta conversaciones multi-turn, integración con múltiples proveedores, y composición programática de flujos de ataque complejos; ideal para red teaming de sistemas agénticos con múltiples herramientas.
- **Garak (NVIDIA, open-source):** más de 100 probes predefinidas organizadas en categorías; ejecución desde CLI contra cualquier endpoint LLM; reporte estructurado con tasa de éxito por probe; ideal para coverage de amenazas conocidas y comparación entre versiones.
- **PAIR (Chao et al., 2023):** LLM attacker (GPT-4) itera sobre prompts adversariales refinando basado en el resultado del intento anterior; 60-80% de éxito en 20 iteraciones; el LLM attacker es a la vez el generador y el evaluador del ataque.
- **TAP (Mehrotra et al., 2023):** extensión de PAIR con tree search y beam pruning que reduce queries necesarios 4-10x; el árbol de ataques permite explorar el espacio de prompts adversariales más eficientemente que la búsqueda lineal de PAIR.
- **Scorer como componente crítico:** el scorer que evalúa si un output es una vulnerabilidad exitosa es el componente más determinante de la calidad del ejercicio automatizado; scorers imprecisos producen falsos positivos (reportan vulnerabilidades donde no las hay) o falsos negativos (miss vulnerabilidades reales); los scorers más efectivos combinan clasificadores fine-tuned con LLM-as-Judge.

Nota del Arquitecto: *La trampa más frecuente con red teaming automatizado es confundir el volumen de intentos con cobertura real de amenazas. Ejecutar 10,000 variaciones de un único tipo de ataque no es lo mismo que cubrir el threat model del sistema. El red teaming automatizado bien diseñado empieza por el threat model — cuáles son las categorías de riesgo más relevantes para este sistema específico— y usa las herramientas para cubrir exhaustivamente esas categorías, no para maximizar el número de intentos.*

El red teaming automatizado no reemplaza al red teaming manual sino que lo complementa: los sistemas automatizados proveen cobertura exhaustiva de variaciones de ataques conocidos a escala, liberando al equipo humano para enfocarse en las vulnerabilidades creativas y contextualmente sofisticadas que la automatización no puede descubrir.

Módulo 9 – Capítulo 04 – Sección 05

Integración en el SDLC: cuándo y con qué frecuencia hacer red teaming

El red teaming de IA no es un evento único que se realiza antes del primer lanzamiento del sistema y luego se archiva como requisito cumplido. Es un proceso continuo que debe integrarse en el Software Development Lifecycle (SDLC) del sistema de IA con la misma naturalidad que las pruebas de seguridad en el desarrollo de software convencional: SAST en cada pull request, DAST en el ambiente de staging antes de cada deploy, penetration testing antes de releases mayores. La frecuencia y profundidad del red teaming debe calibrarse no según un calendario fijo sino según los cambios en el sistema, porque cada cambio introduce potencialmente nuevas vulnerabilidades que no existían antes de ese cambio.

La relación entre capacidad del sistema y superficie de ataque es directamente proporcional: más capacidades significa más superficie que testear. Un cambio de modelo base de GPT-3.5 a GPT-4o no solo mejora la calidad de las respuestas —cambia el comportamiento adversarial del sistema de maneras que no pueden predecirse sin exploración activa. GPT-4o tiene capacidades de visión que GPT-3.5 no tenía, lo que introduce vectores de inyección de instrucciones en imágenes que son completamente nuevos. La adición de una herramienta de búsqueda web a un agente que antes solo consultaba un vectorstore interno introduce la superficie de indirect injection via páginas web maliciosas. La expansión del corpus RAG con documentos de fuentes externas no verificadas amplía la superficie de RAG poisoning. Cada uno de estos cambios invalida parcialmente el threat model anterior y requiere red teaming enfocado en las nuevas superficies.

El modelo operacional de integración en el SDLC tiene tres capas. La primera es el red teaming ligero automatizado en el pipeline de CI/CD: ejecutar un subset de probes de Garak y de PyRIT como gate del proceso de deployment para detectar regresiones de seguridad conocidas. Este layer no es un red teaming completo —no descubrirá vulnerabilidades nuevas — pero es un mecanismo de regresión que previene que vulnerabilidades ya conocidas y ya mitigadas reaparezcan en una nueva versión del sistema. La segunda capa es el red teaming completo pre-release: un ejercicio de dos a cuatro semanas que combina red teaming manual

con red teaming automatizado completo, produciendo un reporte con todos los hallazgos, sus severidades, y las mitigaciones implementadas y verificadas antes del go-live. La tercera capa es el red teaming periódico en producción: trimestral como línea de base, o triggereado por incidentes de seguridad, que incorpora el análisis de logs de producción para identificar attempts de ataque reales (intentos fallidos que no fueron exitosos pero sí detectados) como guía para el threat model actualizado.

La gestión de los hallazgos del red teaming con un proceso disciplinado es lo que convierte los hallazgos en mejoras reales del sistema. Cada vulnerabilidad encontrada debe tener un owner asignado, un SLA de mitigación basado en severidad (crítico: 24 horas; alto: 7 días; medio: 30 días; bajo: 90 días), un plan de mitigación específico, y un re-test con el prompt exacto del hallazgo original para verificar que la mitigación fue efectiva. Sin este proceso, los hallazgos del red teaming se convierten en un inventario de vulnerabilidades conocidas que no se resuelven y que incrementan el riesgo del sistema en lugar de reducirlo.

Aspectos técnicos de integración en SDLC

- **Triggers de red teaming mandatorio:** cambio de modelo base, cambio significativo en el system prompt, adición o modificación de herramientas en agentes, expansión del corpus RAG con fuentes externas no verificadas, habilitación de multimodalidad, cambio en el perfil de usuarios del sistema.
- **Red teaming ligero en CI/CD:** subset de probes de Garak + PyRIT como gate del pipeline de deployment; detecta regresiones de seguridad de vulnerabilidades ya conocidas; no descubre vulnerabilidades nuevas pero previene retrocesos.
- **Red teaming completo pre-release:** ejercicio de 2-4 semanas combinando red teaming manual (expertos especializados) y automatizado (PyRIT + Garak + PAIR); documentación completa de hallazgos y mitigaciones verificadas antes del go-live.
- **Red teaming periódico en producción:** trimestral como mínimo, o triggereado por incidentes; incluye análisis de logs de producción para identificar attempts de ataque reales como guía para el threat model actualizado.
- **SLA de mitigación por severidad:** crítico 24h, alto 7 días, medio 30 días, bajo 90 días; cada hallazgo con owner asignado, plan de mitigación, y re-test de verificación con el prompt original del hallazgo.

El red teaming debe tratarse como un proceso de ingeniería con artefactos reproducibles: los prompts de ataque, los resultados, las mitigaciones aplicadas y los re-tests deben versionarse

en el repositorio del proyecto para tener una historia auditable de la evolución del posture de seguridad del sistema a lo largo del tiempo.

Módulo 9 – Capítulo 04 – Sección 06

Cierre: el red teaming es el único método que descubre vulnerabilidades antes que los atacantes

La diferencia entre un sistema de IA que fue atacado en producción y uno que no lo fue frecuentemente no es la ausencia de vulnerabilidades sino la ausencia de un adversario que las buscó activamente. Los sistemas que no hacen red teaming no son seguros; son afortunados —o aún no han sido descubiertos por el adversario correcto. La razón por la que el red teaming es el mecanismo más efectivo de mejora de la seguridad de sistemas de IA es que responde a esta asimetría: pone al sistema en el rol del adversario antes de que el adversario real llegue, en condiciones controladas donde los hallazgos producen mejoras en lugar de incidentes.

El conocimiento que produce el red teaming tiene un valor que trasciende el ejercicio individual. Los hallazgos de un red teaming no solo mejoran el sistema actual: acumulan en una base de conocimiento de técnicas de ataque que informa el diseño de sistemas futuros, el training de nuevos modelos, y la construcción de guardrails más efectivos. Un equipo que ha realizado red teaming sistemático a lo largo del ciclo de vida de varios sistemas tiene una ventaja epistémica significativa sobre uno que no lo ha hecho: sabe qué tipos de ataques son más probables para su dominio de aplicación, qué técnicas de jailbreak son efectivas contra los modelos que usa, y qué mitigaciones funcionan realmente versus cuáles son efectivas en el papel pero eludibles en la práctica.

La madurez de un equipo de AI Engineering se mide, entre otras métricas, por la sistematicidad con la que integra red teaming en su ciclo de desarrollo. Los equipos inmaduros hacen red teaming reactivo: después de un incidente en producción, o cuando se aproxima un deadline de compliance que lo exige. Los equipos maduros hacen red teaming proactivo: como gate obligatorio antes de cada release significativo, como proceso periódico en producción, y como disciplina de diseño que informa las decisiones arquitectónicas desde el inicio. La diferencia en posture de seguridad entre estos dos tipos de equipos, medida en tiempo hasta detección de vulnerabilidades y en severidad de incidentes en producción, es sistemáticamente favorable al equipo que integra el red teaming como práctica continua.

Conceptos clave del capítulo

- **Red teaming de IA:** evaluación adversarial estructurada que busca vulnerabilidades de seguridad, safety, privacidad y alineación desde la perspectiva del atacante; cubre lo que el testing funcional no puede anticipar.
- **Metodología:** scoping del threat model + harm taxonomy + ejecución con criterios de éxito claros + reporte con hallazgos reproducibles + seguimiento de mitigaciones con SLA por severidad.
- **Red teaming manual:** equipos interdisciplinarios (seguridad, psicología, dominio, idiomas) para vulnerabilidades que requieren creatividad adversarial, conocimiento cultural y adaptabilidad en tiempo real.
- **Red teaming automatizado:** PyRIT (Microsoft) para orquestación multi-turn y conversaciones complejas, Garak (NVIDIA) para cobertura de probes conocidas, PAIR/TAP para generación iterativa con LLM-as-Attacker.
- **Integración en SDLC:** red teaming ligero en CI/CD (gate de regresión), ejercicio completo pre-release (2-4 semanas), red teaming periódico en producción (trimestral mínimo), triggereado por cambios de modelo, system prompt o herramientas.

"Only an attacker who has actually tried to break a system can know how robust it really is." — Ross Anderson, profesor de seguridad de la Universidad de Cambridge y autor de "Security Engineering", sobre el principio fundamental que hace al red teaming insustituible: la única forma de saber si un sistema resiste ataques reales es intentar atacarlo realmente.